

Practical Exam Questions Summary

Problem 1: Pop Middle

Simple Answer: Use doubly linked list (`std::list`) with middle iterator tracking

```
#include <iostream>
#include <list>
#include <string>
using namespace std;

int main() {
    int Q;
    cin >> Q;
    list<int> queue;
    auto middle = queue.end();

    for (int i = 0; i < Q; i++) {
        string operation;
        cin >> operation;

        if (operation == "PUSH") {
            int x;
            cin >> x;
            queue.push_back(x);

            // Update middle
            if (queue.size() == 1) {
                middle = queue.begin();
            } else if (queue.size() % 2 == 1) {
                middle++;
            }
        } else if (operation == "POP") {
            cout << queue.front() << endl;

            // Update middle before removal
            if (queue.size() % 2 == 0) {
                if (queue.begin() == middle) middle++;
            } else {
                if (queue.begin() != middle) middle--;
            }

            queue.pop_front();
        } else if (operation == "POPMIDDLE") {
            cout << *middle << endl;

            auto to_remove = middle;
            if (queue.size() % 2 == 1) {
```

```

        middle--;
    } else {
        middle++;
    }
    queue.erase(to_remove);
}
}

// Output remaining elements
auto it = queue.begin();
if (it != queue.end()) {
    cout << *it;
    it++;
    for (; it != queue.end(); it++) {
        cout << " " << *it;
    }
}
cout << endl;

return 0;
}

```

Key Insight: Maintain iterator to middle element and update it during operations

Problem 2: Speakers

Simple Answer: Use set to store non-overlapping intervals

```

#include <iostream>
#include <set>
#include <string>
using namespace std;

struct Interval {
    long long start, end;
    Interval(long long s, long long e) : start(s), end(e) {}

    bool operator<(const Interval& other) const {
        return start < other.start;
    }
};

int main() {
    long long Q, L;
    cin >> Q >> L;

    set<Interval> schedule;

    for (long long i = 0; i < Q; i++) {
        string op;
        long long S;
        cin >> op >> S;
    }
}

```

```

long long E = S + L - 1;

if (op == "INSERT") {
    bool canInsert = true;

    // Check next interval
    auto it = schedule.lower_bound(Interval(S, E));
    if (it != schedule.end() && it->start <= E) {
        canInsert = false;
    }

    // Check previous interval
    if (it != schedule.begin()) {
        auto prev = it;
        prev--;
        if (prev->end >= S) {
            canInsert = false;
        }
    }

    if (canInsert) {
        cout << "Y" << endl;
        schedule.insert(Interval(S, E));
    } else {
        cout << "N" << endl;
    }
} else if (op == "REMOVE") {
    auto it = schedule.find(Interval(S, S + L - 1));
    if (it != schedule.end() && it->start == S) {
        cout << "Y" << endl;
        schedule.erase(it);
    } else {
        cout << "N" << endl;
    }
}

// Output final schedule
bool first = true;
for (const auto& interval : schedule) {
    if (!first) cout << " ";
    cout << interval.start;
    first = false;
}
cout << endl;

return 0;
}

```

Key Insight: Store intervals in sorted set, check neighbors for overlap

Problem 3: Queens

Simple Answer: Backtracking with column/diagonal tracking

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

vector<string> board;
vector<bool> col_used, diag1_used, diag2_used;
int N, M;

bool canPlace(int row, int col) {
    if (col_used[col]) return false;
    if (diag1_used[row + col]) return false;
    if (diag2_used[row - col + N]) return false;
    return true;
}

bool backtrack(int row, int queens_placed) {
    if (queens_placed == N) return true;
    if (row == N) return false;

    // Check if current row already has a queen
    for (int col = 0; col < N; col++) {
        if (board[row][col] == 'Q') {
            // This row already has queen, move to next
            return backtrack(row + 1, queens_placed + 1);
        }
    }

    // Try placing queen in each column
    for (int col = 0; col < N; col++) {
        if (canPlace(row, col)) {
            col_used[col] = true;
            diag1_used[row + col] = true;
            diag2_used[row - col + N] = true;

            if (backtrack(row + 1, queens_placed + 1)) {
                return true;
            }

            col_used[col] = false;
            diag1_used[row + col] = false;
            diag2_used[row - col + N] = false;
        }
    }

    // Try skipping this row
    return backtrack(row + 1, queens_placed);
}
```

```

int main() {
    cin >> N >> M;
    board.resize(N);

    col_used.assign(N, false);
    diag1_used.assign(2 * N, false);
    diag2_used.assign(2 * N, false);

    int existing_queens = 0;
    for (int i = 0; i < N; i++) {
        cin >> board[i];
        for (int j = 0; j < N; j++) {
            if (board[i][j] == 'Q') {
                existing_queens++;
                col_used[j] = true;
                diag1_used[i + j] = true;
                diag2_used[i - j + N] = true;
            }
        }
    }

    if (backtrack(0, existing_queens)) {
        cout << "true" << endl;
    } else {
        cout << "false" << endl;
    }

    return 0;
}

```

Key Insight: Track attacked columns and diagonals, use backtracking to place remaining queens

Problem 4: Instruction

Simple Answer: Recursion to try all combinations of applying instructions

```

#include <iostream>
#include <vector>
#include <string>
#include <climits>
#include <cmath>
using namespace std;

int N, K;
vector<pair<char, int>> instructions;
long long best_result = 0;
long long min_diff = LLONG_MAX;

void solve(int idx, long long current) {
    if (idx == N) {
        long long diff = abs(current - K);
    }
}

```

```

        if (diff < min_diff || (diff == min_diff && current < best_resu
            min_diff = diff;
            best_result = current;
        }
        return;
    }

    // Option 1: Skip current instruction
    solve(idx + 1, current);

    // Option 2: Apply current instruction
    long long new_val = current;
    char op = instructions[idx].first;
    int num = instructions[idx].second;

    switch (op) {
        case '+': new_val += num; break;
        case '-': new_val -= num; break;
        case '*': new_val *= num; break;
        case '/': new_val /= num; break;
    }

    solve(idx + 1, new_val);
}

int main() {
    cin >> N >> K;
    instructions.resize(N);

    for (int i = 0; i < N; i++) {
        cin >> instructions[i].first >> instructions[i].second;
    }

    solve(0, 0);
    cout << best_result << endl;

    return 0;
}

```

Key Insight: Try all 2^N combinations using recursion, track best result

Problem 5: Longest Increasing Subsequence

Simple Answer: Brute force all subsets (small $N \leq 10$)

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    int N;

```

```

cin >> N;
vector<int> A(N);

for (int i = 0; i < N; i++) {
    cin >> A[i];
}

int max_len = 0;

// Try all 2^N subsets
for (int mask = 1; mask < (1 << N); mask++) {
    vector<int> subsequence;
    for (int i = 0; i < N; i++) {
        if (mask & (1 << i)) {
            subsequence.push_back(A[i]);
        }
    }

    // Check if increasing
    bool increasing = true;
    for (int i = 1; i < subsequence.size(); i++) {
        if (subsequence[i] <= subsequence[i - 1]) {
            increasing = false;
            break;
        }
    }

    if (increasing) {
        max_len = max(max_len, (int)subsequence.size());
    }
}

cout << max_len << endl;

return 0;
}

```

Alternative $O(N^2)$ DP Solution:

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    int N;
    cin >> N;
    vector<int> A(N), dp(N, 1);

    for (int i = 0; i < N; i++) {
        cin >> A[i];
    }
}

```

```

int max_len = 1;
for (int i = 0; i < N; i++) {
    for (int j = 0; j < i; j++) {
        if (A[j] < A[i]) {
            dp[i] = max(dp[i], dp[j] + 1);
        }
    }
    max_len = max(max_len, dp[i]);
}

cout << max_len << endl;

return 0;
}

```

Key Insight: For small N, brute force all subsets; for larger N, use DP

Summary of Data Structures Used:

- **Problem 1:** list (doubly linked list)
- **Problem 2:** set (BST for intervals)
- **Problem 3:** vector (arrays for backtracking)
- **Problem 4:** vector + recursion
- **Problem 5:** vector + bitmasking or DP